

The pipeline transforms raw historical cryptocurrency data into a predictive model (Random Forest) and deploys it via a Streamlit web interface. The architecture flows sequentially from **Data Ingestion** to **Deployment**.

1. Data Ingestion & Merging

The pipeline begins by collecting raw data snapshots from CoinGecko.

- **Sources:** The system reads two specific CSV files: `coin_gecko_2022-03-16.csv` and `coin_gecko_2022-03-17.csv`.
- **Aggregation:** These individual daily snapshots are concatenated into a single, comprehensive DataFrame to form the historical dataset.

2. Data Preprocessing (Cleaning)

Before the data is usable, it undergoes rigorous cleaning to handle errors and missing information.

- **Deduplication:** Duplicate rows are removed to prevent data leakage or bias.
- **Imputation Strategy:** Missing values in numerical columns (like `price`, `24h_volume`, `mkt_cap`) are handled in two stages:
 1. **Group-Specific Mean:** First, missing values are filled using the mean value specific to that cryptocurrency symbol (e.g., filling a missing Bitcoin price with the average Bitcoin price).
 2. **Global Mean:** Any remaining gaps are filled with the overall column mean as a fallback.

3. Scaling (Normalization)

To ensure the Machine Learning model interprets features correctly, the data is standardized.

- **Technique:** A `StandardScaler` is used to normalize numerical features (mean = 0, variance = 1).
- **Artifact Preservation:** Crucially, this scaler fits on the training data and is saved as `feature_scaler.pkl`. This allows the Streamlit app to scale new user inputs exactly the same way the model expects.

4. Feature Engineering (Time-Series Lagging)

This step converts the dataset into a supervised learning problem.

- **Target Creation (\$Y\$):** The goal is to predict *future* liquidity. The system creates a new target variable, `target_liquidity_volume`, by shifting the `24h_volume` column backwards by one day (using `.shift(-1)`). This aligns "Today's Metrics" with "Tomorrow's Volume".

- **Feature Selection (\$X\$):** The features used for prediction include `price`, `1h`, `24h`, `7d` (percentage changes), `24h_volume`, and `mkt_cap`.

5. Model Training & Evaluation

- **Algorithm:** A **Random Forest Regressor** is selected for its robustness and ability to handle non-linear relationships.
- **Hyperparameter Tuning:** The model is tuned using `RandomizedSearchCV` to find the optimal number of trees (`n_estimators`) and tree depth (`max_depth`).
- **Serialization:** The trained model is saved as `final_rf_model.pkl` for deployment.

6. Deployment (Streamlit App)

The final stage is the user-facing application (`app.py`), which loads the saved artifacts to make real-time predictions.

- **Artifact Loading:** The app loads the pre-trained `final_rf_model.pkl` and `feature_scaler.pkl`.
- **Input Processing:** User inputs (Price, Volume, Market Cap, etc.) are collected via the UI and scaled using the loaded `StandardScaler`.
- **Inference & Inverse Transformation:** The model predicts a *scaled* liquidity value (Z-score). The app mathematically reverses the scaling (Inverse Transform) to convert the Z-score back into a readable USD dollar amount.

